

**Пермский институт (филиал) федерального государственного
бюджетного образовательного учреждения высшего
образования
«Российский экономический университет имени Г.В.
Плеханова»**

Практическая работа #1

Тема работы: Создание и запуск модульных тестов для
управляемого кода

Работу выполнила:
Пономарева Ангелина
Алексеевна
Группа: ИПо-21
Преподаватель: Берестов
Дмитрий Борисович

Пермь 2026

СОДЕРЖАНИЕ

Создание проекта для тестирования.....	3
Создаем проект модульного теста.....	6
Создаем тестовый класс.....	8
Создаем первый тестовый метод.....	10
Сборка и запуск теста.....	11
Исправляем код и делаем повторный запуск тестов.....	12
Заключение.....	17

Создание проекта для тестирования

Первое что я сделаю это создам проект для тестирования и назову его своей фамилией Ponomareva.

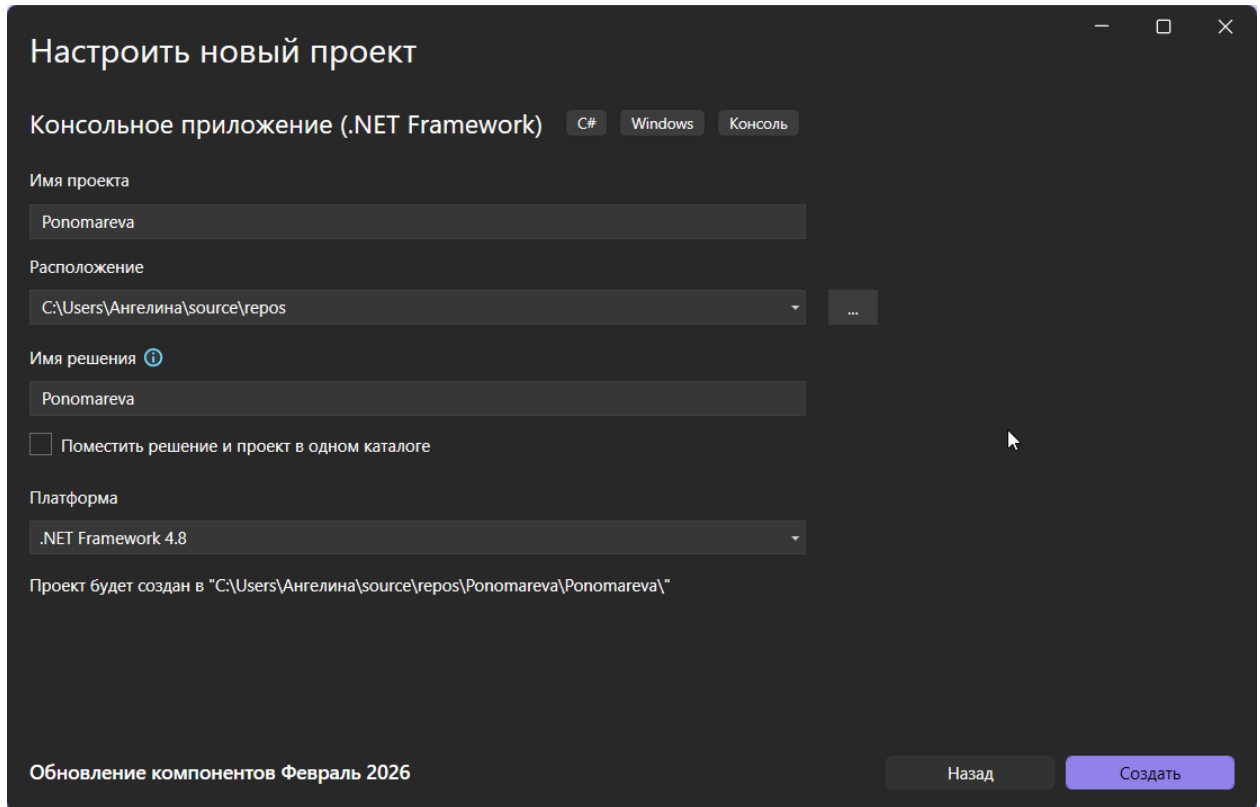


Рис.1 - Создание проекта

После создания файла требуется изменить существующий код. Нужный фрагмент представлен на рисунке 2 и отвечает за определение класса BankAccount.

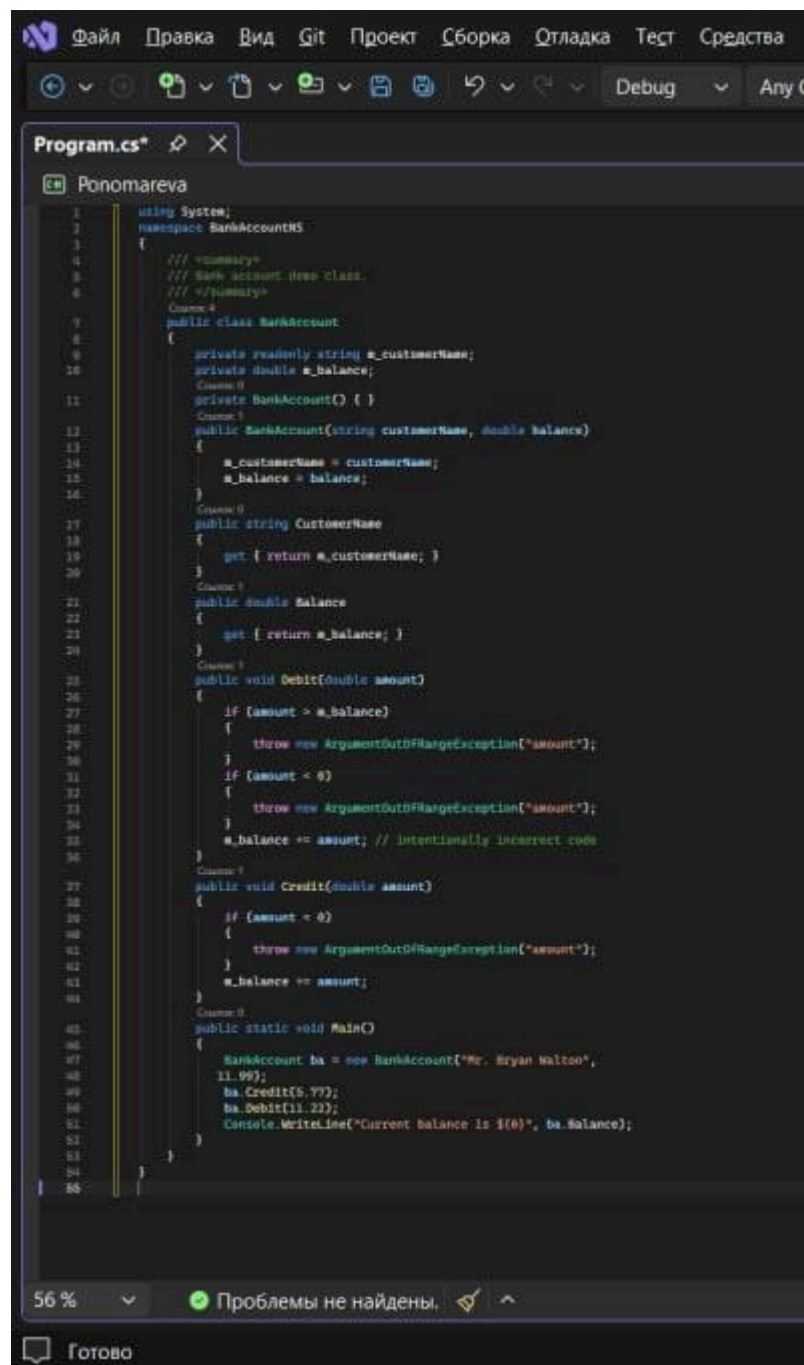


Рис. 2 - Замена кода

После замены кода на требуемый, переходим к переименованию файла в «BankAccount.cs». Далее выполняем сборку проекта, нажимая комбинацию клавиш «CTRL + SHIFT + B».

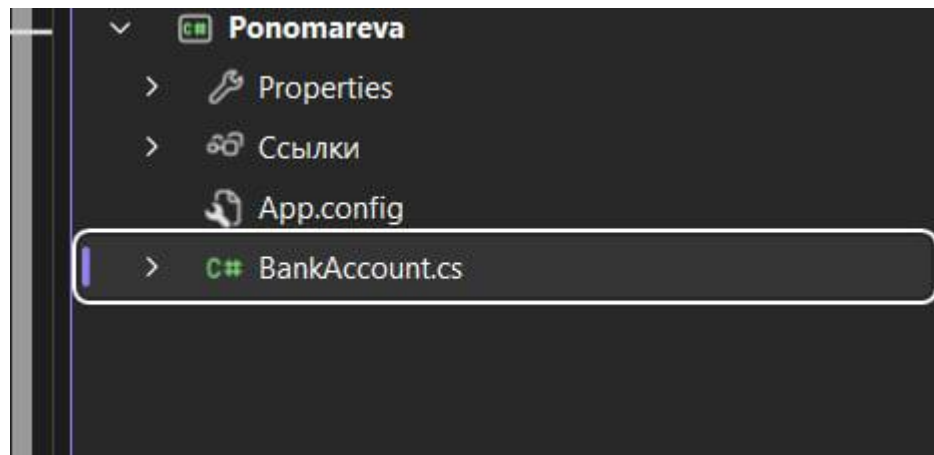


Рис. 3 - Переименовываем файл

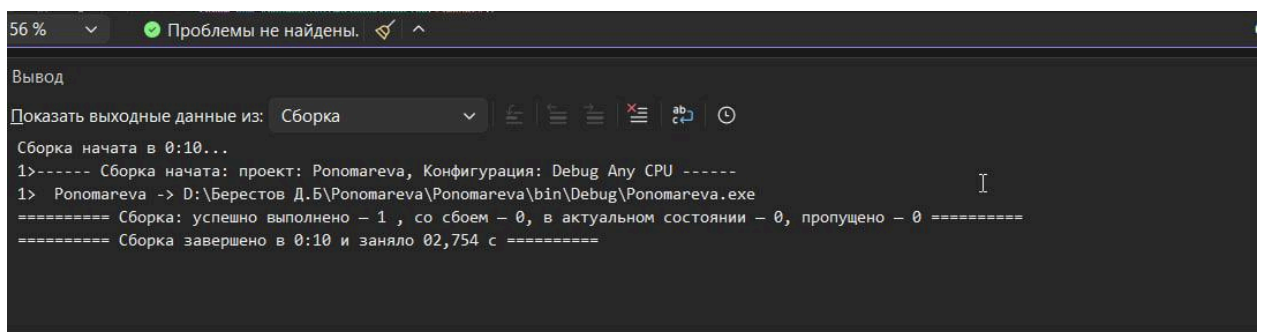


Рис. 4 - Сборка

Этим завершается процесс подготовки тестового файла, далее приступаем к созданию проекта для модульного тестирования.

Создаем проект модульного теста

Первым делом создаем новый проект , добавив его в наше решение. Называю его PonomarevaTests.

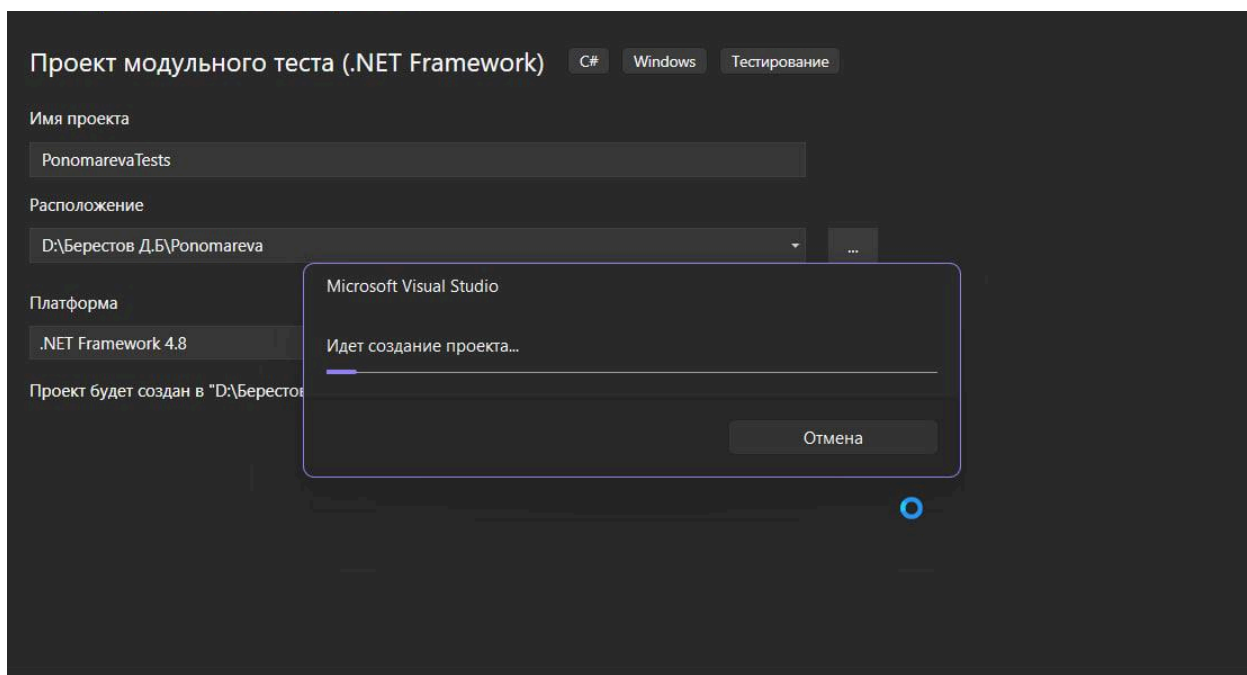


Рис. 5 - Создание нового проекта

Следующим шагом я установила зависимость между проектами. Для этого в Обозревателе решений я нажала правой кнопкой мыши на пункт "Ссылки" в проекте PonomarevaTests, выбрала команду "Добавить ссылку на проект" и добавила проект "Ponomareva"

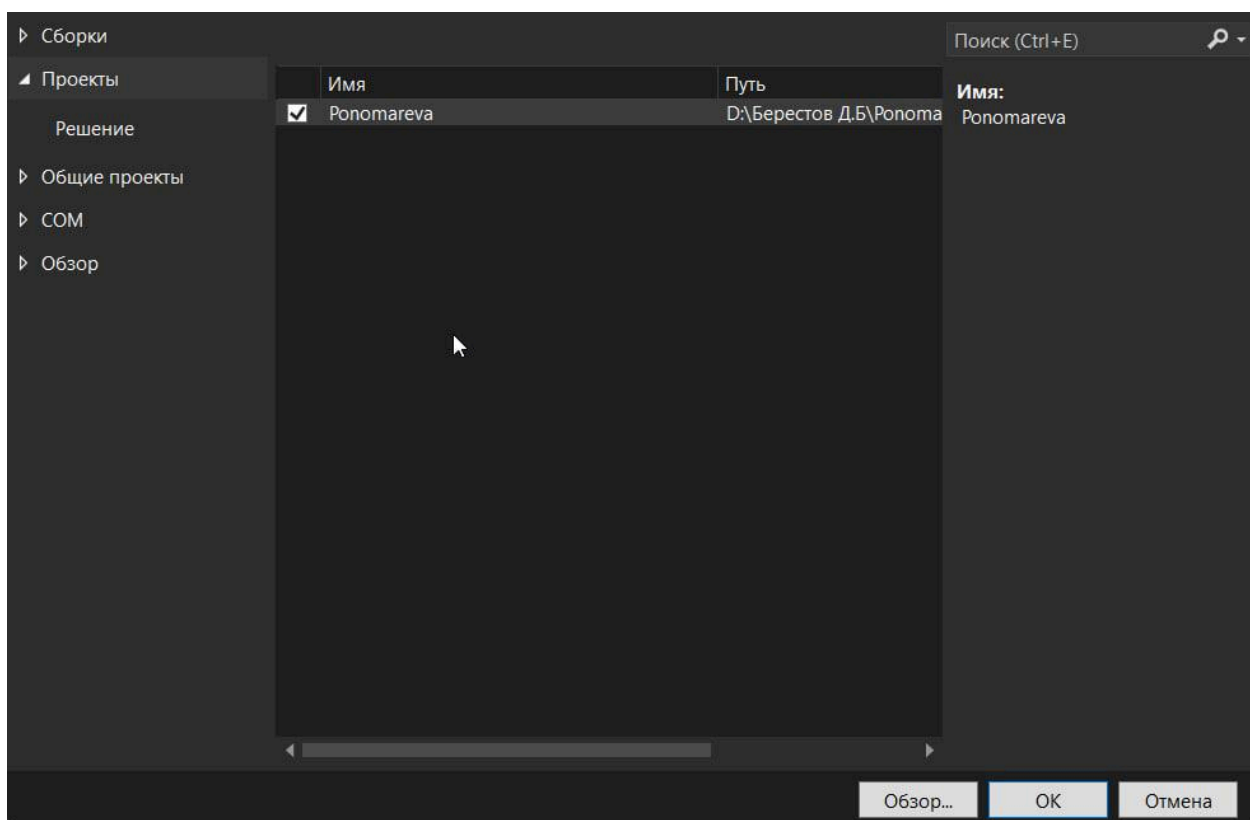


Рис. 6 - Прикрепление ссылки

Создаем тестовый класс

Переименовываю файл и класс. Меняем название с «UnitTest1.cs» на «BankAccountTests.cs».

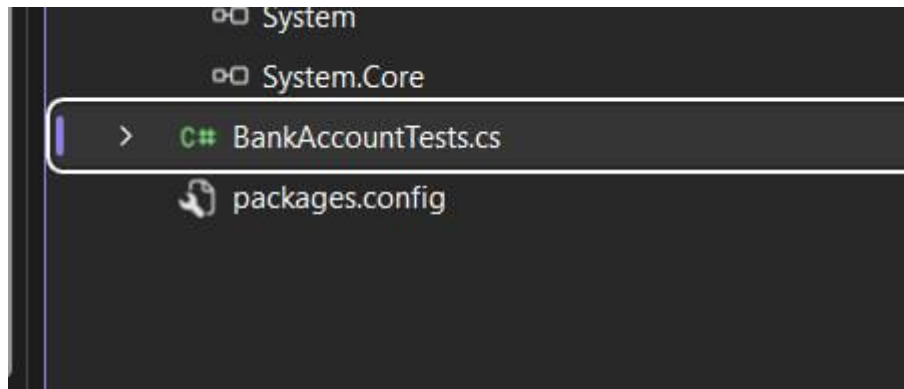


Рис. 7 - Переименовываем UnitTest1.cs

Теперь файл BankAccountTests.cs содержит такой код:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;  
using BankAccountNS;
```

```
namespace BankTests  
{  
    [TestClass]  
    public class BankAccountTests  
    {  
        [TestMethod]  
        public void TestMethod1()  
        {  
        }  
    }  
}
```

В начале файла тестового класса я прописала оператор using BankAccountNS;. Данное действие необходимо для того, чтобы иметь возможность вызывать методы основного проекта без необходимости каждый раз указывать их полное имя.


```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using BankAccountNS;

namespace BankTests
{
    [TestClass]
    Ссылка: 0
    public class BankAccountTests
    {
        [TestMethod]
        Ссылка: 0
        public void TestMethod1()
        {
        }
    }
}
```

Рис. 8 - Добавление оператора using

Создаем первый тестовый метод

Для проверки базового функционала я добавила в класс BankAccountTests метод, тестирующий снятие допустимой суммы. Цель этого теста — убедиться, что при списании суммы (больше нуля и меньше баланса) итоговый остаток на счете рассчитывается верно.

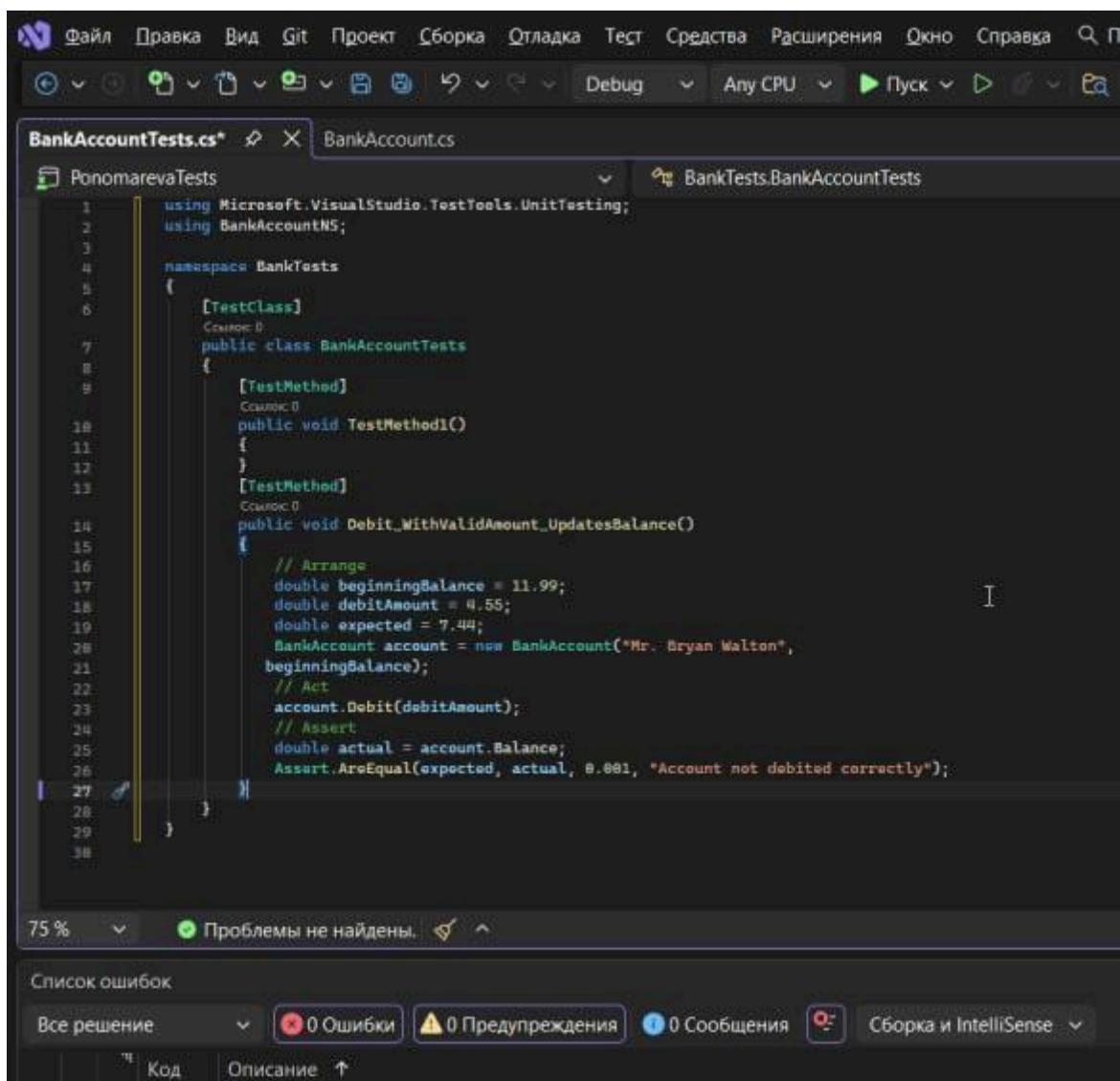


Рис. 9 - Код первого теста

Принцип работы этого метода заключается в следующем: сначала создается объект счета с начальным балансом, после чего выполняется операция списания допустимой суммы. Результат проверяется методом Assert.AreEqual — он подтверждает, что итоговый остаток совпал с ожидаемым.

Сборка и запуск теста

После написания кода я выполнил сборку решения через меню "Сборка" (Ctrl + Shift + B). Затем я открыл окно "Обозреватель тестов", чтобы запустить проверку. Нажав кнопку "Запустить все", я инициировал выполнение теста. В этом случае тест пройден не будет.

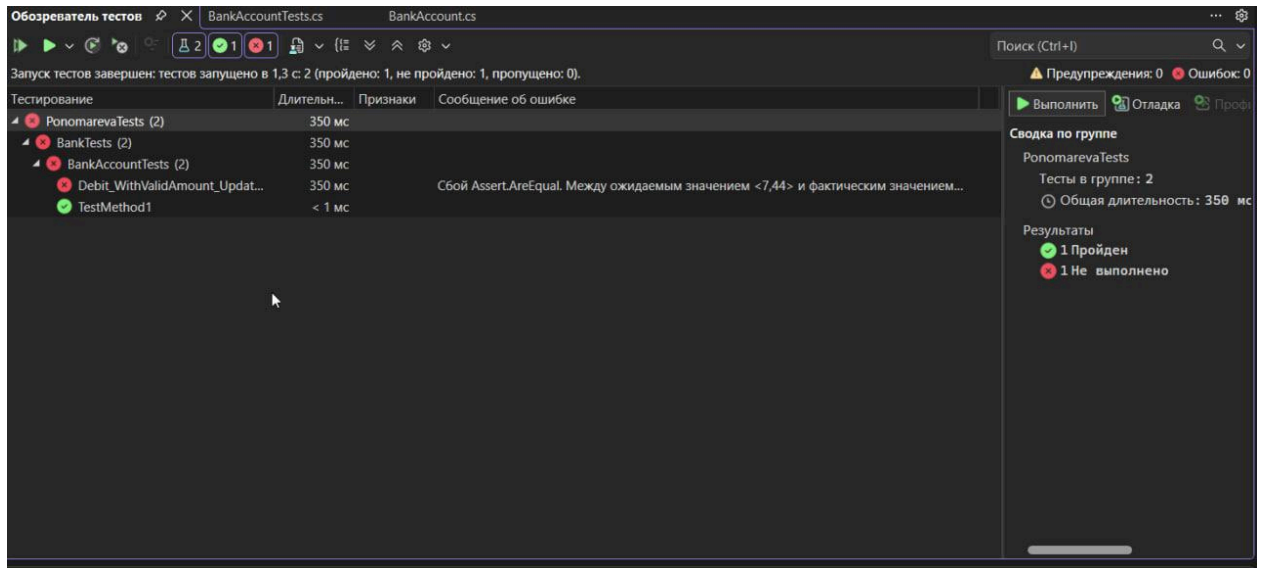


Рис. 10 - Результат теста

Завершающим этапом я запустила общее тестирование системы. Результат проверяется по цветовому индикатору: зелёный цвет означает успех, красный — наличие ошибки. В данном случае тест провалился (загорелся красным), что подтвердило наличие бага в логике: программа ошибочно прибавляла сумму к балансу вместо её вычитания.

Исправляем код и делаем повторный запуск тестов

Для устранения выявленного бага я внесла изменения в исходный код файла BankAccount.cs, заменив некорректную строку на правильную.

```
    }  
    m_balance -= amount; // intentionally incorrect code  
}  
  
Ссылка 1  
public void Credit(double amount)  
{  
    if (amount < 0)  
    {  
        throw new ArgumentOutOfRangeException("amount");  
    }  
    m_balance -= amount;  
}  
  
Ссылка 0  
public static void Main()
```

Рис. 11 - Исправляем код

После внесения правок я перешла в "Обозреватель тестов" и повторно запустил проверку кнопкой "Запустить все". Индикатор состояния, который ранее был красным, сменился на зеленый. Это подтвердило, что ошибка успешно исправлена и теперь тест пройден без замечаний.

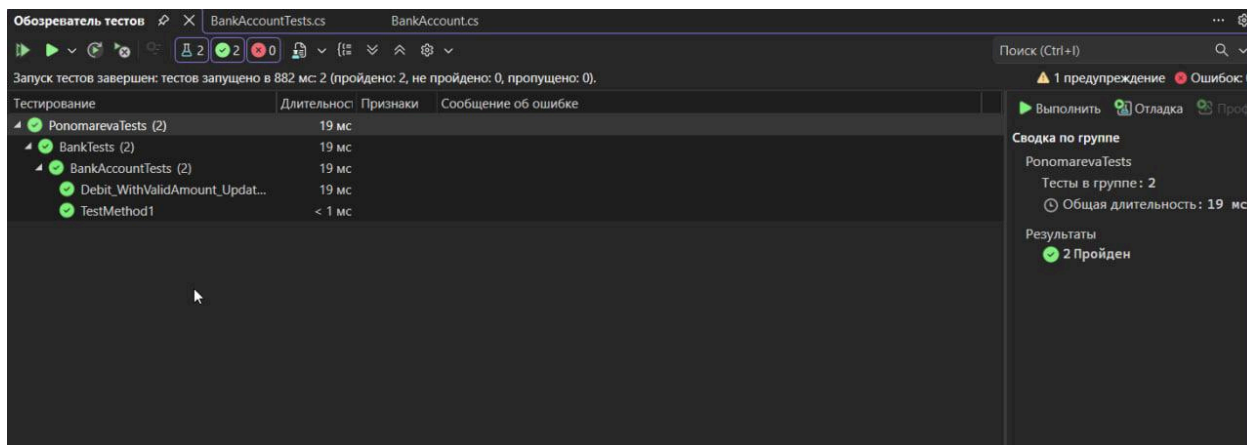


Рис. 12 - Повторное проведение теста

Создам метод теста для проверки правильного поведения в случае, когда сумма по дебету меньше нуля.

```

    {
        // Arrange
        double beginningBalance = 11.99;
        double debitAmount = 4.55;
        double expected = 7.44;
        BankAccount account = new BankAccount("Mr. Bryan Walton",
        beginningBalance);
        // Act
        account.Debit(debitAmount);
        // Assert
        double actual = account.Balance;
        Assert.AreEqual(expected, actual, 0.001, "Account not debited correctly");
    }
    [TestMethod]
    Ссылка 0
    public void Debit_WhenAmountIsLessThanZero_ShouldThrowArgumentOutOfRangeException()
    {
        // Arrange
        double beginningBalance = 11.99;
        double debitAmount = -100.00;
        BankAccount account = new BankAccount("Mr. Bryan Walton",
        beginningBalance);
        // Act and assert
        Assert.ThrowsException<System.ArgumentOutOfRangeException>(() =>
        account.Debit(debitAmount));
    }
}

```

Рис. 13 - Код нового теста

Метод `ThrowsException` проверяет, возникает ли нужное исключение при выполнении определенного участка кода. В нашем примере мы ожидаем, что появится именно исключение типа `ArgumentOutOfRangeException`, когда сумма по дебету меньше нуля.

```

}
[TestMethod]
Ссылка 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.00;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);
    // Act and assert
    Assert.ThrowsException<System.ArgumentOutOfRangeException>(() =>
    account.Debit(debitAmount));
}

```

Рис. 14 - Код теста с превышающей баланс суммой

Задаём переменной `debitAmount` значение, превышающее баланс. Теперь можно запускать тесты.

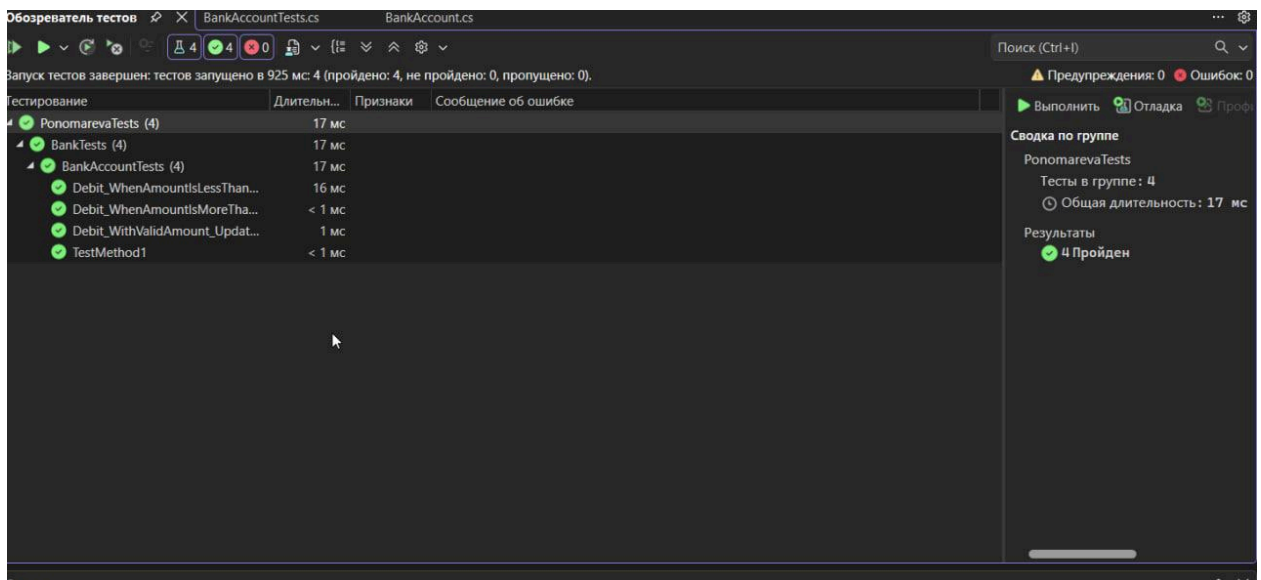


Рис. 15 - Статус тестов

Все тесты успешно пройдены, однако их можно улучшить. Усовершенствование позволит точнее проверять корректность обработки аргументов методом и правильность формируемых исключений.



Рис. 16 - Добавляем две константы

Следующим шагом я внес изменения в логику метода Debit, обновив два условных оператора проверки входных данных.



Рис. 17 - Изменяем два условных оператора

Теперь приступаю к рефакторингу тестового кода. Это позволит значительно улучшить диагностику: теперь в случае неудачи я смогу видеть конкретную причину сбоя теста, а не просто сам факт ошибки.


```

[TestMethod]
Ссылка 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton", beginningBalance);
    // Act
    try
    {
        account.Debit(debitAmount);
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(e.Message, BankAccount.DebitAmountExceedsBalanceMessage);
    }
}

```

Рис. 18 - Рефакторинг кода методов теста

Этот тест подвергся изменениям. Сначала убрали проверку на возникновение исключений с помощью метода `Assert.ThrowsException`. Затем вызов метода `Debit()` поместили внутрь блока `try/catch`.

```

[TestMethod]
Ссылка 0
public void Debit_WhenAmountIsMoreThanBalance_ShouldThrowArgumentOutOfRangeException()
{
    // Arrange
    double beginningBalance = 11.99;
    double debitAmount = 20.0;
    BankAccount account = new BankAccount("Mr. Bryan Walton",
beginningBalance);
    // Act
    try
    {
        account.Debit(debitAmount);
    }
    catch (System.ArgumentOutOfRangeException e)
    {
        // Assert
        StringAssert.Contains(e.Message,
BankAccount.DebitAmountExceedsBalanceMessage);
        return;
    }
    Assert.Fail("The expected exception was not thrown.");
}
}

```

Рис. 19 - Доделанный рефакторинг кода методов теста

Первым делом добавили оператор `return`. Его задача — предотвратить аварийное завершение теста, которое могло бы произойти из-за нового утверждения `Assert.Fail`. Само утверждение `Assert.Fail` используется специально для обработки ситуаций, когда ожидалось появление исключения, однако оно не возникло.

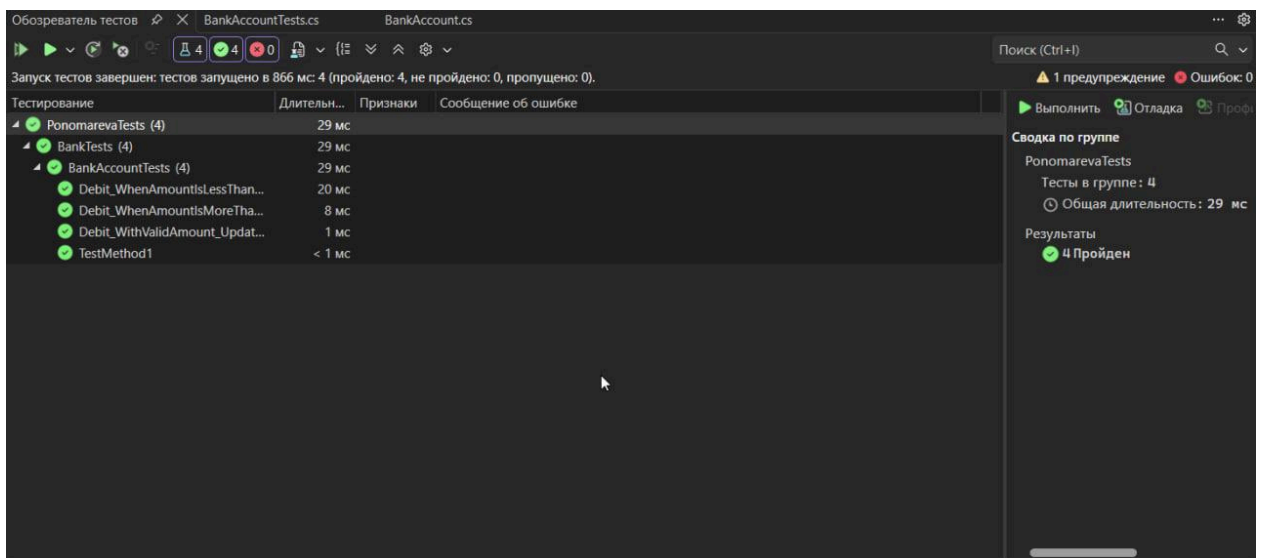


Рис. 20 - Проведение тестов

Заключение

В ходе выполнения данной практической работы я освоила создание консольных приложений и написание модульных тестов на платформе .NET Framework. Дополнительно научилась добавлять новые проекты в существующий проект и улучшать качество исходного кода путем написания надежных тестов. Таким образом, данная практика помогла значительно расширить мои компетенции в разработке ПО на платформе .NET Framework.